

Atividades Práticas com Diagramas de Caso de Uso

Aplicando conceitos de UML em cenários do mundo real

Nesta aula, aplicaremos os conceitos de UML através da modelagem de sistemas diversos utilizando diagramas de caso de uso. Exploraremos cenários práticos como:

1 Sistemas Bancários

Modelagem de operações financeiras, perfis de usuários e fluxos de segurança

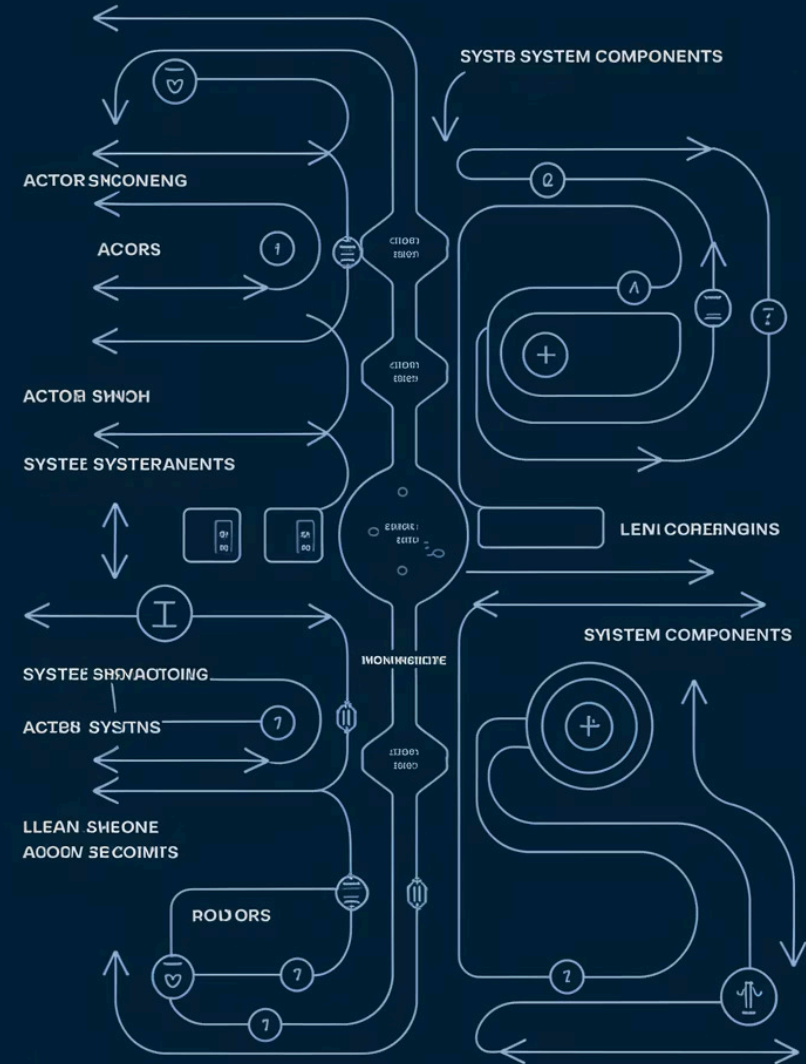
2 Aplicativos de Transporte (Uber)

Interações entre passageiros, motoristas e sistema de pagamento

3 Sistemas de RH e Financeiros

Gestão de funcionários, folha de pagamento e operações de investimento

Cada exemplo permitirá que você compreenda na prática como estruturar diagramas eficientes, identificar atores e casos de uso, e traduzir requisitos em representações visuais que orientarão o desenvolvimento de software.





Modelagem de Sistemas Bancários com Diagramas de Caso de Uso

Esta apresentação abordará a modelagem de sistemas bancários através de diagramas de caso de uso. Exploraremos como representar atores e suas interações com o sistema. O objetivo é entender e visualizar as funcionalidades essenciais de um sistema bancário.

G por Giovanni Amorim

Clientes: Consultar, Transferir, Pagar

Consultar Saldo

Clientes verificam o saldo da sua conta bancária.

Realizar Transferência

Clientes podem transferir fundos para outras contas.

Pagar Boletos

Clientes efetuam o pagamento de boletos online.

Gerentes: Abrir e Fechar Contas



Abrir Conta

O gerente pode abrir novas contas bancárias para os clientes.



Fechar Conta

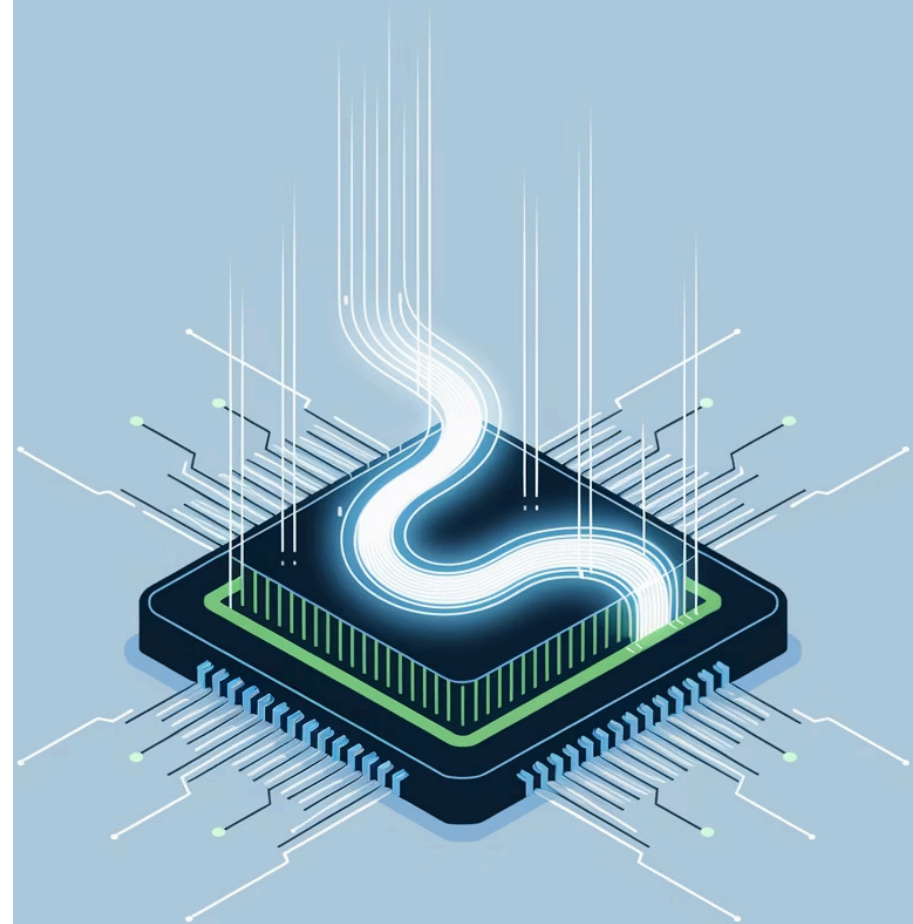
O gerente pode fechar contas de clientes mediante solicitação.



Sistema de Segurança: Autenticação

1 Autenticar Usuário

Validação de credenciais para acesso ao sistema.



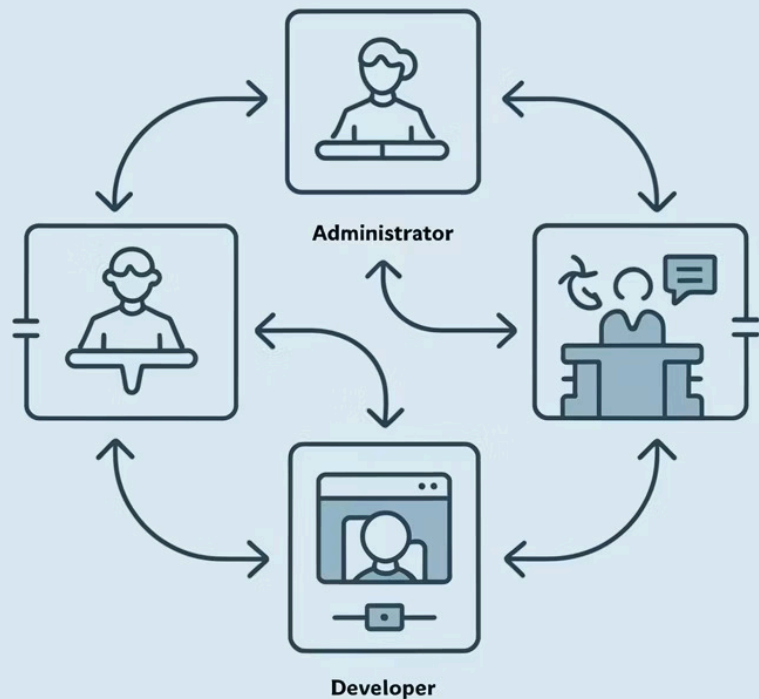


Diagrama de Caso de Uso: Atores

1

Cliente

Realiza operações financeiras.

2

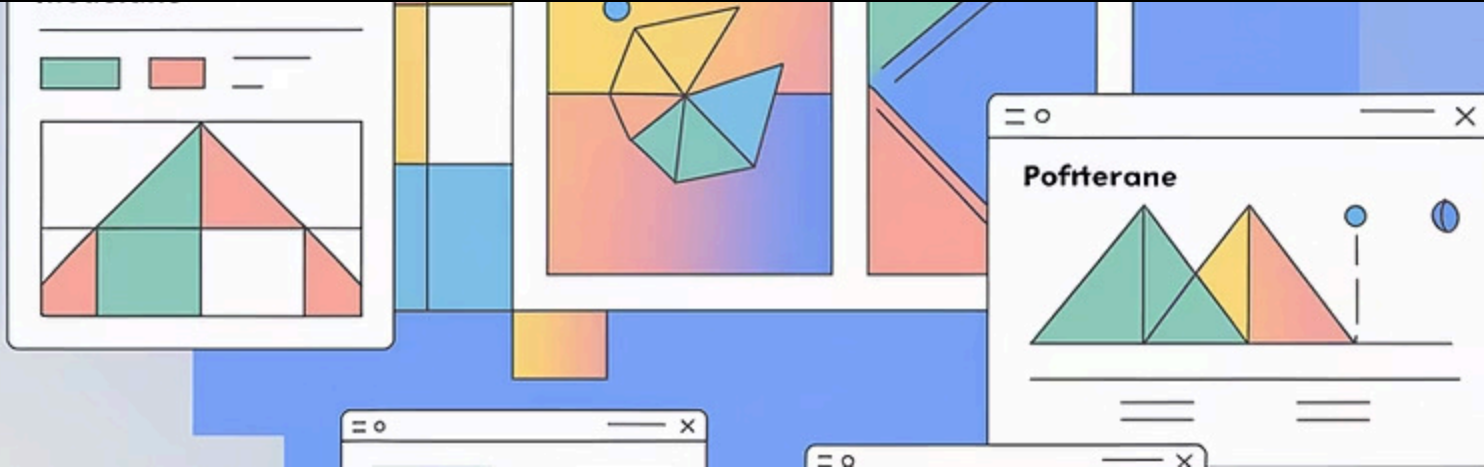
Gerente

Gerencia contas e serviços.

3

Sistema de Segurança

Proteção contra fraudes.



Visual Paradigm: Ferramenta Essencial

Modelagem Visual

Crie diagramas intuitivos.

Colaboração

Trabalhe em equipe.

Documentação

Gere relatórios detalhados.

Atividade Prática: Sistema Bancário

1

Definir Atores

2

Casos de Uso

3

Interações

Utilize Visual Paradigm para criar o diagrama de caso de uso. Inclua todos os atores, casos de uso e interações. Este é o sistema bancário.

Atividade 2: Sistema de Uber

Objetivo: Desenvolver um Diagrama de Caso de Uso para o sistema do Uber, representando os atores e suas interações com os principais casos de uso do sistema.

Contexto: O sistema do Uber permite que passageiros solicitem corridas e paguem por elas, enquanto motoristas aceitam corridas e recebem pagamentos.



Identificar os Atores

Passageiros, motoristas, sistema de pagamento, etc.



Definir os Casos de Uso

Solicitar corrida, aceitar corrida, realizar pagamento, avaliar serviço, etc.



Estabelecer Relacionamentos

Conexões entre os atores e casos de uso no diagrama.

Utilize o Visual Paradigm para criar o diagrama completo com todos os elementos necessários.

Sistema de RH

Objetivo: Desenvolver um Diagrama de Caso de Uso para um sistema de Recursos Humanos (RH), representando os atores e suas interações com os principais casos de uso do sistema.

Contexto: O Sistema de RH permite que funcionários realizem consultas sobre suas férias e atualizem seus dados cadastrais. Os gestores podem avaliar o desempenho dos funcionários e aprovar solicitações de férias. O sistema também se comunica com um Sistema Externo de Holerite.



Funcionários

Consultam saldo de férias e atualizam dados cadastrais no sistema.



Gestores

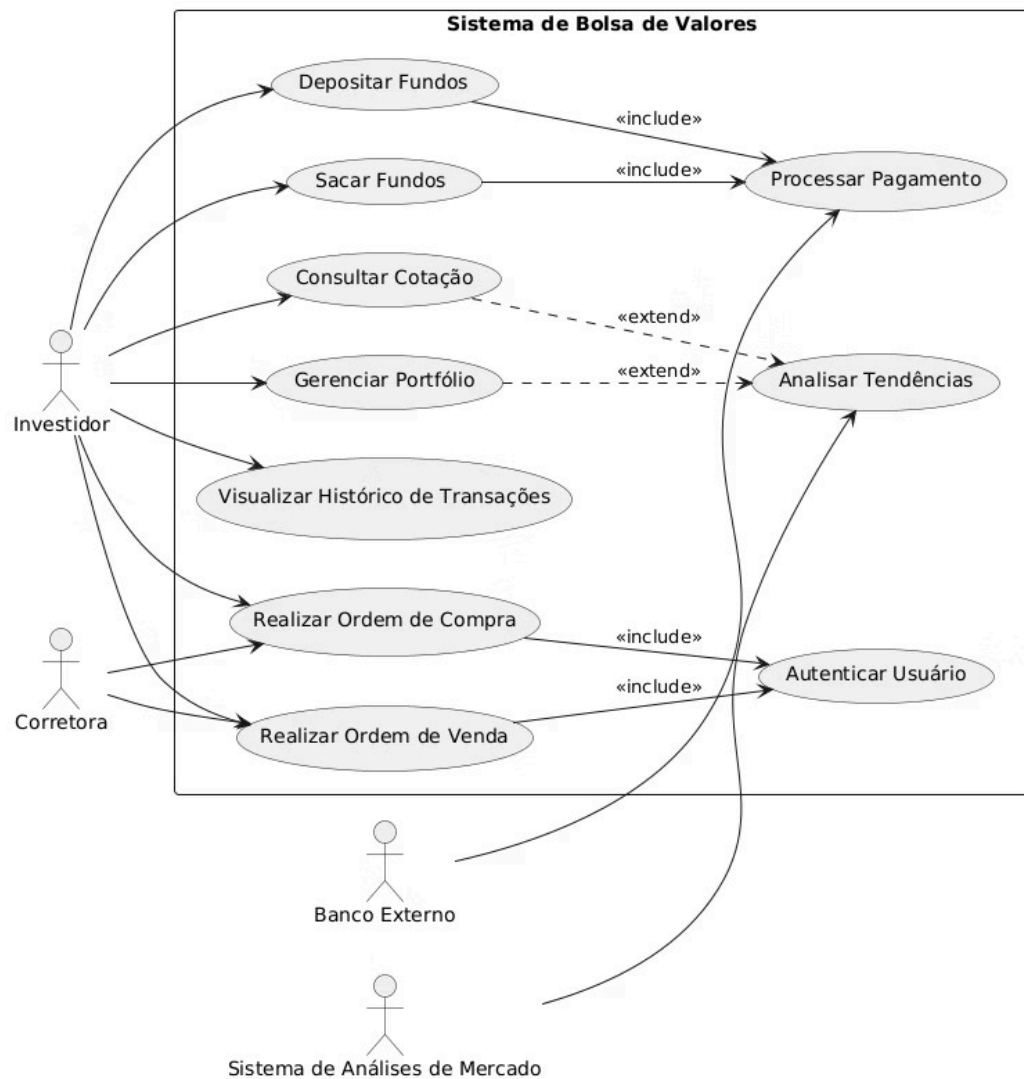
Avaliam desempenho dos funcionários e aprovam solicitações de férias.



Sistema Externo

Integração com o Sistema Externo de Holerite para processamento de informações.

Utilize o Visual Paradigm para criar o diagrama completo com todos os elementos necessários.



Liste os seguintes itens:

- **Atores Primários** (quem interage diretamente com o sistema).
- **Atores Secundários** (sistemas externos ou suporte ao funcionamento).
- **Sistema** (o que está sendo modelado).
- **Casos de Uso** (as funcionalidades principais do sistema).

Descreva cada caso de uso respondendo às perguntas:

- Qual é o nome do caso de uso?
- Quem pode executá-lo?
- Qual é o objetivo dessa funcionalidade?
- Existe alguma dependência com outro caso de uso?

Identifique dois erros no caso de uso e sugira a correção.

Código Java de um Sistema Financeiro

Classe Principal do Sistema Financeiro

```
import java.util.ArrayList;
import java.util.List;

public class SistemaFinanceiro {
    private List transacoes;
    private double saldo;

    public SistemaFinanceiro() {
        this.transacoes = new ArrayList<>();
        this.saldo = 0.0;
    }

    public void adicionarReceita(double valor, String descricao) {
        transacoes.add(new Transacao(valor, descricao, TipoTransacao.RECEITA));
        saldo += valor;
        System.out.println("Receita adicionada com sucesso!");
    }

    public void adicionarDespesa(double valor, String descricao) {
        transacoes.add(new Transacao(valor, descricao, TipoTransacao.DESPESA));
        saldo -= valor;
        System.out.println("Despesa adicionada com sucesso!");
    }

    public void visualizarTransacoes() {
        System.out.println("\nLista de Transações:");
        for (Transacao t : transacoes) {
            System.out.println(t);
        }
        System.out.println("Saldo Atual: R$ " + saldo);
    }

    public void gerarRelatorio() {
        double totalReceitas = 0, totalDespesas = 0;
        for (Transacao t : transacoes) {
            if (t.getTipo() == TipoTransacao.RECEITA) {
                totalReceitas += t.getValor();
            } else {
                totalDespesas += t.getValor();
            }
        }
        System.out.println("\nRelatório Financeiro:");
        System.out.println("Total de Receitas: R$ " + totalReceitas);
        System.out.println("Total de Despesas: R$ " + totalDespesas);
        System.out.println("Saldo Final: R$ " + saldo);
    }
}
```

Classe Main do Sistema Financeiro

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        SistemaFinanceiro sistema = new SistemaFinanceiro();

        while (true) {
            System.out.println("\n=== Menu ===");
            System.out.println("1. Adicionar Receita");
            System.out.println("2. Adicionar Despesa");
            System.out.println("3. Visualizar Transações");
            System.out.println("4. Gerar Relatório Financeiro");
            System.out.println("5. Sair");
            System.out.print("Escolha uma opção: ");
            int opcao = scanner.nextInt();
            scanner.nextLine(); // Consumir nova linha

            switch (opcao) {
                case 1:
                    System.out.print("Digite o valor da receita: ");
                    double receita = scanner.nextDouble();
                    scanner.nextLine();
                    System.out.print("Digite a descrição: ");
                    String descricaoReceita = scanner.nextLine();
                    sistema.adicionarReceita(receita, descricaoReceita);
                    break;
                case 2:
                    System.out.print("Digite o valor da despesa: ");
                    double despesa = scanner.nextDouble();
                    scanner.nextLine();
                    System.out.print("Digite a descrição: ");
                    String descricaoDespesa = scanner.nextLine();
                    sistema.adicionarDespesa(despesa, descricaoDespesa);
                    break;
                case 3:
                    sistema.visualizarTransacoes();
                    break;
                case 4:
                    sistema.gerarRelatorio();
                    break;
                case 5:
                    System.out.println("Encerrando o sistema...");
                    scanner.close();
                    return;
                default:
                    System.out.println("Opção inválida, tente novamente.");
            }
        }
    }
}
```

Classe para Representar uma Transação Financeira

```
class Transacao {
    private double valor;
    private String descricao;
    private TipoTransacao tipo;

    public Transacao(double valor, String descricao, TipoTransacao tipo) {
        this.valor = valor;
        this.descricao = descricao;
        this.tipo = tipo;
    }

    public double getValor() {
        return valor;
    }

    public TipoTransacao getTipo() {
        return tipo;
    }

    @Override
    public String toString() {
        return tipo + ": " + descricao + " - R$ " + valor;
    }
}
```

Enum para Definir o Tipo da Transação

```
enum TipoTransacao {
    RECEITA, DESPESA;
}
```